

16. ETAPA 11 — Documentación técnica

Nombre del proyecto

SIGED — Sistema Integral de Gestión de Emergencias y Desastres

Descripción

SIGED es un sistema web que desarrollamos para organizar la información relacionada con la atención de emergencias y desastres. El proyecto surgió a partir de la idea de tener en un mismo sistema la información que normalmente podría estar repartida entre documentos, hojas de cálculo o diferentes registros. El sistema permite registrar incidentes como inundaciones, incendios, terremotos y otros tipos de emergencias. También permite consultar el historial de cambios de cada incidente, administrar refugios y recursos, asignar personal y consultar las alertas generadas dentro del sistema.

Una de las partes importantes del proyecto es el manejo de roles. Dependiendo del rol que tenga el usuario, puede acceder a diferentes opciones del sistema. Los roles utilizados son Administrador, Rescatista, Gestor de Refugio y Voluntario, por ejemplo, un usuario con el rol de Voluntario no necesita tener acceso a la administración de usuarios, mientras que un Rescatista puede consultar los incidentes y su historial. De esta manera, las opciones que aparecen en el sistema dependen de las funciones que corresponde realizar a cada usuario.

Problema

Durante la planificación del proyecto identificamos que manejar información de emergencias en diferentes lugares puede provocar problemas para encontrar datos, actualizar información o conocer el estado actual de una situación, en un escenario de emergencia es necesario saber qué incidentes están activos, qué recursos están disponibles, qué refugios existen y qué personas están asignadas. Cuando esta información se encuentra separada, puede ser más difícil mantenerla actualizada por esta razón decidimos desarrollar un sistema que reuniera esta información en una sola aplicación y que además permitiera controlar el acceso de los usuarios.

Justificación

Elegimos desarrollar SIGED porque nos permitía aplicar varios de los temas que trabajamos durante el curso en un mismo proyecto, durante el desarrollo utilizamos conocimientos de bases de datos, desarrollo de APIs, Angular, autenticación, manejo de roles y conexión entre Frontend y Backend, también nos pareció un tema adecuado porque permitía trabajar con diferentes tipos de información y relaciones entre tablas. Por ejemplo, un incidente puede tener varios registros en su historial, mientras que los recursos y el personal pueden

relacionarse con diferentes situaciones dependiendo de las necesidades, además de cumplir con los requisitos del proyecto integrador, el sistema nos permitió practicar el proceso completo de desarrollo, desde el diseño de la base de datos hasta la creación de las pantallas y las pruebas de funcionamiento.

Objetivos

Objetivo general

Desarrollar un sistema web para centralizar la gestión de incidentes, recursos, refugios y personal relacionados con la atención de emergencias, utilizando control de acceso según el rol de cada usuario.

Objetivos específicos

- Registrar, consultar, editar y eliminar.
- Mantener un historial de los cambios realizados en los incidentes.
- Administrar los refugios registrados.
- Administrar los recursos disponibles.
- Permitir asignar recursos y personal.
- Registrar y consultar alertas.
- Controlar las opciones disponibles según el rol del usuario.
- Mantener protegidas las diferentes secciones del sistema.
- Facilitar la consulta de información desde una sola aplicación.

Requerimientos

Requerimientos funcionales

El sistema cuenta con las siguientes funciones principales:

- Inicio de sesión mediante correo y contraseña.
- Registro de usuarios.
- Manejo de diferentes roles.
- Creación, consulta, edición y eliminación.
- Registro del historial de cambios de estado de los incidentes.
- Administración de refugios.

- Administración de recursos.
- Administración de zonas.
- Asignación de recursos.
- Asignación de personal.
- Creación y consulta de alertas.
- Visualización de diferentes opciones del menú dependiendo del rol.

Requerimientos no funcionales

- El sistema debe funcionar en navegadores web modernos.
- Las pantallas deben adaptarse a diferentes tamaños de pantalla.
- Las contraseñas no deben almacenarse directamente en texto plano.
- El acceso a los módulos debe estar protegido.
- El sistema debe responder correctamente ante errores de comunicación con el Backend.
- La información debe mantenerse organizada entre el Frontend, Backend y Base de Datos.

Arquitectura

Para desarrollar SIGED dividimos el sistema en tres partes principales: Frontend, Backend y Base de Datos.

Frontend

El Frontend fue desarrollado con Angular y TypeScript. En esta parte se encuentran las pantallas que utiliza el usuario, como el inicio de sesión, dashboard, incidentes, historial, logística, operaciones y usuarios, también se implementaron componentes y servicios para separar las diferentes responsabilidades de la aplicación.

Backend

El Backend fue desarrollado con Node.js, Express y TypeScript, su función principal es recibir las solicitudes realizadas desde el Frontend, validar la información recibida, comprobar los permisos del usuario y comunicarse con PostgreSQL, de esta manera el Frontend no se conecta directamente con la base de datos.

Base de Datos

La base de datos se desarrolló utilizando PostgreSQL, en ella se almacenan los usuarios, incidentes, historial de incidentes, zonas, refugios, recursos, alertas y las diferentes asignaciones.

Tecnologías utilizadas

- **Angular:** utilizado para desarrollar el Frontend.
- **TypeScript:** utilizado tanto en el Frontend como en el Backend.
- **Node.js:** utilizado para ejecutar el Backend.
- **Express:** utilizado para crear la API.
- **PostgreSQL:** utilizado para almacenar la información.
- **JWT:** utilizado para manejar la autenticación.
- **bcryptjs:** utilizado para proteger las contraseñas.
- **Angular Material:** utilizado en diferentes elementos de la interfaz.
- **pnpm:** utilizado para administrar las dependencias del Backend.
- **npm:** utilizado para administrar las dependencias del Frontend.
- **Git:** utilizado para llevar el control de versiones.

Estas tecnologías fueron seleccionadas principalmente porque eran compatibles con los conocimientos trabajados durante el curso y porque permitían separar correctamente las diferentes partes del sistema.

Base de datos

La base de datos está formada por varias tablas relacionadas entre sí.

Las principales son:

- **usuario:** contiene la información de los usuarios y su rol.
- **incidente:** almacena los incidentes registrados.
- **historial_incidente:** almacena los cambios de estado de los incidentes.
- **zona:** contiene las zonas utilizadas dentro del sistema.
- **refugio:** almacena la información de los refugios.
- **recurso:** contiene los recursos disponibles.

- **asignacion_recurso:** relaciona los recursos con los incidentes o refugios.
- **asignacion_personal:** relaciona usuarios con zonas o incidentes.
- **alerta:** almacena las alertas generadas.

Una relación importante es la existente entre incidente e historial_incidente. Cada registro del historial corresponde a un incidente específico y permite consultar cómo fue cambiando su estado.

Funcionamiento general

El funcionamiento comienza cuando el usuario ingresa al sistema mediante la pantalla de inicio de sesión, después de validar las credenciales, se genera la sesión correspondiente y el usuario puede acceder al panel principal, las opciones disponibles dependen del rol asignado cuando el usuario realiza una acción, por ejemplo crear un incidente, Angular envía la información al servicio correspondiente. El servicio realiza la petición al Backend y el Backend valida los datos y los permisos antes de realizar la operación en PostgreSQL, después de completar la operación, el Backend devuelve una respuesta al Frontend y Angular actualiza la información que se muestra en pantalla.

Funcionamiento del historial

El historial se utiliza para mantener un registro de los cambios de estado de un incidente, por ejemplo, un incidente puede comenzar con el estado REPORTADO y posteriormente pasar a EN_ATENCION, MITIGADO y finalmente CERRADO.

Cada cambio puede almacenar información como:

- Incidente relacionado.
- Estado anterior.
- Estado nuevo.
- Comentario.
- Usuario relacionado.
- Fecha del cambio.

En el Frontend se desarrollaron diferentes vistas para consultar esta información, incluyendo la lista de registros, el detalle y la visualización del historial de manera cronológica.

Pruebas

Durante el desarrollo realizamos diferentes pruebas para comprobar que los módulos funcionaran correctamente.

Se realizaron pruebas de:

- Inicio de sesión.
- Registro de usuarios.
- Control de acceso según el rol.
- Creaciones.
- Ediciones.
- Eliminaciones.
- Consultas.
- Administración de recursos.
- Administración de refugios.
- Asignación de personal.
- Consulta de alertas.
- Validación de formularios.

También comprobamos que las opciones del menú cambiaran dependiendo del usuario que iniciara sesión.

Errores encontrados

Durante el desarrollo encontramos diferentes problemas que tuvimos que revisar y corregir, uno de los problemas que tuvimos fue con la vista de Historial. Los registros sí existían en la base de datos, pero no se mostraban correctamente en el Frontend. Primero comprobamos que la información estuviera almacenada y posteriormente revisamos el servicio utilizado para obtener los datos, después de revisar la comunicación entre el Frontend y el Backend encontramos el problema relacionado con el consumo de la respuesta del servicio. Una vez corregido, volvimos a cargar la vista y comprobamos que los registros aparecieran correctamente.

También encontramos problemas relacionados con la validación de algunos formularios. En determinadas pruebas era posible intentar guardar información incompleta. Para evitar

esto agregamos validaciones en el Frontend y comprobaciones adicionales en el Backend, estos problemas fueron útiles para entender que no basta con comprobar solamente si una pantalla funciona; también es necesario revisar la comunicación entre los diferentes componentes del sistema.

Mantenimiento

Para mantener el sistema funcionando correctamente recomendamos:

- Revisar periódicamente los registros del Backend.
- Mantener actualizadas las dependencias.
- Realizar respaldos de la base de datos.
- Documentar las nuevas funciones que se agreguen.
- Revisar los permisos cuando se agreguen nuevos roles.
- Realizar pruebas después de modificar módulos importantes.

Conclusiones

El desarrollo de SIGED nos permitió poner en práctica diferentes conocimientos adquiridos durante el curso, uno de los aspectos que más trabajamos fue la comunicación entre el Frontend, el Backend y la base de datos. Al principio cada parte podía funcionar por separado, pero al integrarlas fue necesario revisar que las peticiones, respuestas y datos coincidieran correctamente, también aprendimos que desarrollar un sistema no consiste solamente en crear las pantallas. Fue necesario trabajar con validaciones, autenticación, permisos, relaciones entre tablas y manejo de errores.

Durante el desarrollo tuvimos problemas, principalmente relacionados con algunas vistas y con la forma en que se manejaban algunos datos. Revisar estos errores nos permitió entender mejor cómo funcionaba cada parte del proyecto, al finalizar conseguimos integrar los principales módulos de SIGED y establecer una estructura que permite continuar agregando nuevas funciones posteriormente.

Recomendaciones

Acciones recomendadas:

Antes de instalar el sistema recomendamos revisar las versiones de Node.js, PostgreSQL y Angular que se están utilizando. Si las versiones son diferentes a las que necesita el proyecto, pueden aparecer problemas al instalar las dependencias o al ejecutar algunas partes del sistema. Los usuarios que vienen incluidos en el seed son principalmente para

hacer pruebas durante el desarrollo. Si el sistema se utiliza en un ambiente real, sería mejor cambiar sus contraseñas o crear nuevos usuarios con contraseñas más seguras. También recomendamos hacer respaldos de la base de datos antes de realizar cambios importantes. Así, si algo llega a salir mal, se puede recuperar la información. Cuando se quiera probar una función nueva, es mejor hacerlo primero en el ambiente de desarrollo y después pasarla al sistema que ya está funcionando. Por último, cuando se agregue una función o se haga algún cambio importante, es conveniente dejarlo anotado. También se pueden agregar comentarios en las partes del código que sean más difíciles de entender. Esto ayuda a que los demás integrantes puedan saber qué se modificó si necesitan trabajar nuevamente en esa parte.

Advertencias de seguridad o riesgos:

Uno de los puntos que más debemos cuidar es el archivo `.env`. No se debe subir a GitHub ni compartirlo, porque contiene información como las credenciales de la base de datos y la clave secreta utilizada para la autenticación. También recomendamos no mantener los usuarios de prueba activos cuando el sistema se utilice en un ambiente real, principalmente porque las contraseñas utilizadas durante las pruebas pueden ser demasiado sencillas. Las validaciones del Frontend ayudan a evitar errores cuando el usuario utiliza el sistema, pero no deben ser la única protección. Las peticiones también tienen que validarse en el Backend, ya que un usuario podría intentar enviar una petición directamente sin utilizar los formularios de Angular. Antes de realizar modificaciones directamente en la base de datos es recomendable hacer un respaldo. Un cambio incorrecto podría afectar información que ya se encuentra almacenada. Por último, se debe revisar que las rutas y las funciones que necesitan permisos tengan su respectivo control de roles. Esto ayuda a evitar que un usuario con permisos limitados pueda acceder a información o realizar acciones que no corresponden a su rol.